

Name: Phan Tran Thanh Huy

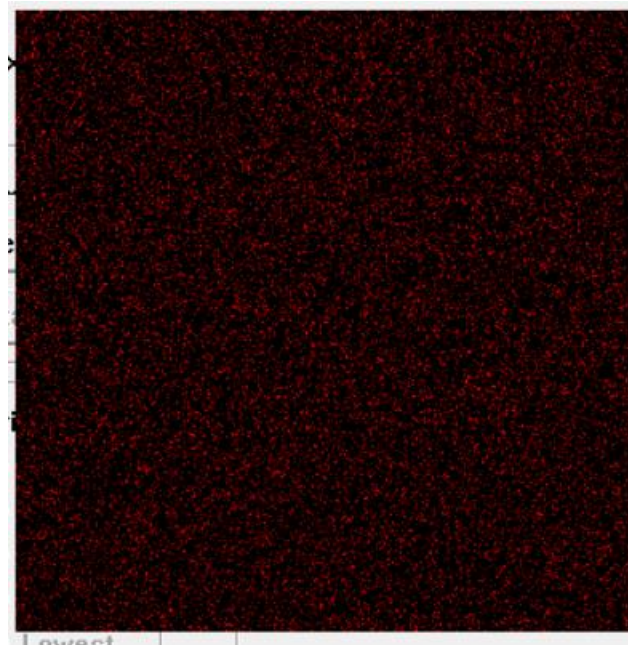
ID: ITCSIU22056

Lab 5

Lab Activity: Threads: Introductory: Single Thread Introduction to the Introductory Threads application

1. In the 'Schedule Threads Asynchronously' section press the Create Thread button for the RED thread.
2. Press the Start Thread button for the RED thread.

Q1. What happens? The screen become red.



3. When the drawing canvas has turned completely red, press the Stop Thread button for the RED thread.

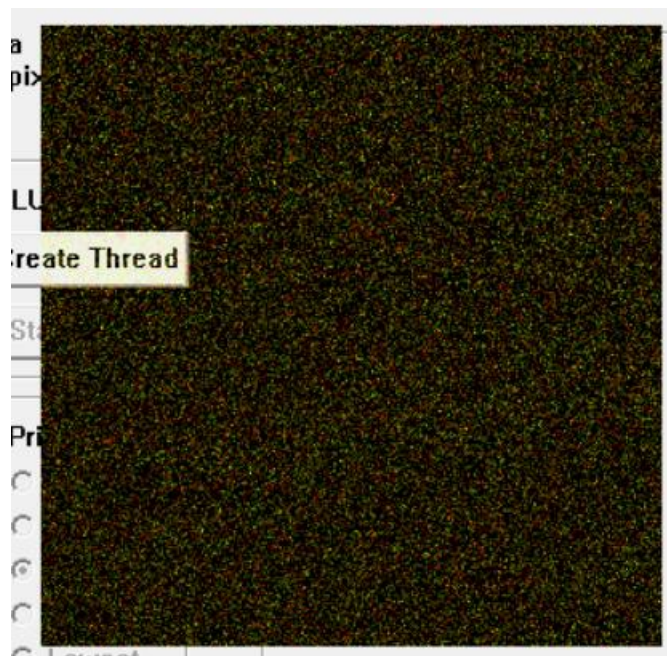
Q2. Why were the pixels not colored evenly (why did some take longer to be colored red than others)? Because the thread selects pixels randomly. Pixels chosen more often increase their red value faster, while those chosen less frequently take longer to change.

Lab Activity: Threads: Introductory: Two Threads Introduction to the Introductory Threads application

Q1. What do you think will happen if two threads run at the same time? The two threads compete to color pixels: RED increases red, GREEN increases green, and each reduces the other colors. The canvas will show a mix of the two colors.

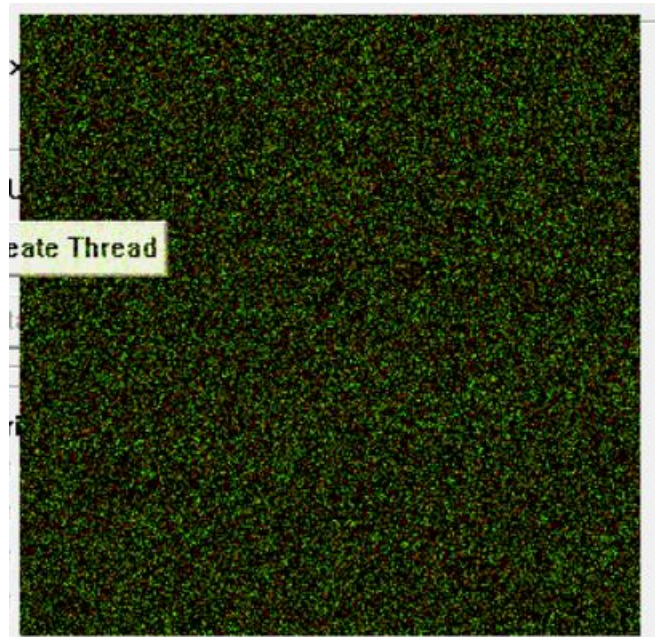
1. In the 'Schedule Threads Asynchronously' section press the Create Thread button for the RED thread.
2. In the 'Schedule Threads Asynchronously' section press the Create Thread button for the GREEN thread.
3. Press the Start Thread button for the RED thread.
4. After a short time (about three seconds) press the Start Thread button for the GREEN thread.

Q2. What happens? The canvas is initially red. When GREEN starts, many pixels shift toward green or yellow/olive due to the mixture of RED and GREEN.



5. After about twenty seconds, press the Stop Thread button for the RED thread.

Q3. What happens? Only the GREEN thread is running, so remaining pixels gradually turn fully green as GREEN colors them.



Lab Activity: Threads: Introductory: Three Threads Introduction to the Introductory Threads application

Q1. What do you think will happen if three threads run at the same time? What colour will the drawing canvas appear to be? With RED, GREEN, and BLUE threads running together, pixels compete among three colors, producing a grayish/washed-out color due to balance among the three.

1. In the 'Schedule Threads Asynchronously' section press the Create Thread button for the RED thread.
2. Press the Start Thread button for the RED thread.
3. In the 'Schedule Threads Asynchronously' section press the Create Thread button for the BLUE thread.
4. In quick succession, Press the Start Thread button for each of the threads.

Q2. What happens? The canvas shows many blended areas; it's unclear which color dominates.

5. After about ten seconds, press the Stop Thread button for the RED thread.

Q2. What happens? RED stops competing → GREEN and BLUE continue; pixels gradually shift to green/blue mix.

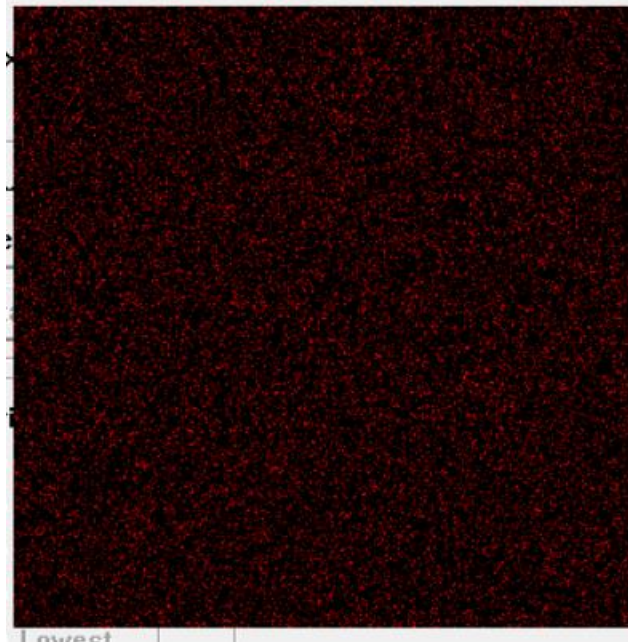


6. After another ten seconds, press the Stop Thread button for the GREEN thread.

Q3. What happens? Only BLUE remains; the entire canvas gradually turns blue.



7. You may, up until this point, have been answering the questions in terms of what is happening visually. Now go back over this activity and consider the questions in terms of the CPU time. Think about how the CPU time is divided between the threads that are running – how much of a share do they each get. **Ans:** CPU time is roughly divided among running threads.
8. Think about the scheduling of the threads – which scheduling algorithm best describes what you see happening when all three threads are running (FCFS, SJF, RR or SRJN)? (Make sure you can justify your answer). **Ans:** The most appropriate scheduling algorithm: Round Robin (RR), because threads get CPU time in turns, not based on length or arrival order.



9. When the drawing canvas has turned completely red, press the Stop Thread button for the RED thread.

Q2. Why were the pixels not colored evenly (why did some take longer to be colored red than others)? Because the thread selects pixels randomly. Pixels chosen more often increase their red value faster, while those chosen less frequently take longer to change.

Lab Activity: Threads: Introductory: Asynchronous vs Synchronous Thread Scheduling

Understanding the differences between Asynchronous and Synchronous Thread Scheduling

Q1. Can you clearly see the difference between the two modes of scheduling in terms of the thread behaviors? To confirm this, think how you would clearly explain what is happening to a friend?

Ans:

- **Asynchronous:** OS schedules threads, interleaving is fine-grained; threads update pixels concurrently.

- **Synchronous:** Threads run one at a time in turns, so you see colors update in “batches.”

Q2. In which mode is it easiest to see that there are separate threads running?

Ans: Synchronous, because each thread runs in sequence, so color changes clearly alternate.

Q3. Which mode would be suitable for an application in which each thread dealt with a different query, such as a web-server? Why?

Ans: Asynchronous, since it allows smooth multi-threading, multiple requests simultaneously, dynamic and flexible CPU allocation.

Lab Activity: Threads: Introductory: Thread Priority

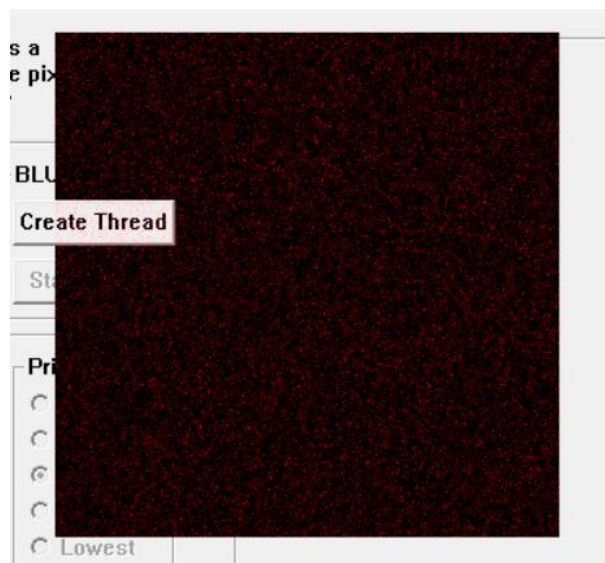
Understanding Thread Priority in both Asynchronous and Synchronous Thread Scheduling

Asynchronous Thread Scheduling.

1. Create and Start three threads. Experiment with various priority configurations.

Q1. What is the visual effect of increasing a thread’s priority?

Ans: Its color appears more on the canvas (dominates) and faster.



Q2. What actually causes this visual effect (in terms of the thread scheduling)?

Ans: Higher priority threads get more CPU time, so they color more pixels.

Q3. What is the effect of uniformly increasing the priority of ALL threads to HIGHEST? What is the effect uniformly decreasing the priority of ALL threads to LOWEST? What is the effect of setting one thread's priority to HIGH and setting the other two threads' priority to LOW? (try also other configurations).

- All High: all threads behave similarly -> Not much different from normal.
- All Low: Similar to normal.
- One High, two Low: High thread almost dominates the canvas.

Q4.

Ans: It is Relative. If all thread change equally, there is no difference.

Synchronous Thread Scheduling.

2. Create and Start the threads. Experiment with various priority configurations, USING THE SCHEDULERS INTERNAL PRIORITY MECHANISM.

Q5. What is the visual effect of increasing a thread's priority? Why is this?

Ans: There is no significant effect. The OS sees only one process, not individual thread priorities.

3. Create and Start the threads. Experiment with various priority configurations, USING EXTERNALLY IMPOSED PRIORITY.

Q6. What is the visual effect of increasing a thread's priority? Why is this?

Ans: The prioritized thread gets more turns; its color dominates the canvas.

Q7. What actually causes this visual effect (in terms of the thread scheduling)?

Ans: The application controls the number of times each thread is invoked, not the OS.

Q8. What is the effect of uniformly increasing the priority of ALL threads to HIGHEST? What is the effect uniformly decreasing the priority of ALL threads to LOWEST? What is the effect of setting one thread's priority to HIGH and setting the other two threads' priority to LOW? (try also other configurations).

- All High/Low: no significant change.
- One High, two Low: High thread runs more and dominates canvas.

Q9.

Ans: It is Relative, same as asynchronous.